

Foundations of Layer normalization

[Layer normalization]

1.0 Content Block

$$\hat{x}_i = \frac{x_i}{\sqrt{\frac{1}{D} \sum_{j=1}^D x_j^2}}, \quad y_i = \gamma \hat{x}_i + \beta \quad \text{where } \gamma, \beta \in \mathbb{R}^D$$

2.0 Content Block

Weight normalization (WeightNorm) is a technique inspired by BatchNorm that normalizes weight matrices in a neural network, rather than its activations. One example is spectral normalization, which divides weight matrices by their spectral norm. The spectral normalization is used in generative adversarial networks (GANs) such as the Wasserstein GAN. The spectral radius can be efficiently computed by the following algorithm:

3.0 Content Block

where B is the batch size, H is the height of the feature map, and W is the width of the feature map. That is, even though there are only B data points in a batch, all BHW outputs from the kernel in this batch are treated equally. Subsequently, normalization and the linear transform is also done per kernel:

4.0 Content Block

where i indexes the coordinates of the vectors, and b indexes the elements of the batch. In other words, we are considering the i -th coordinate of each vector in the batch, and computing the mean and variance of these numbers. It then normalizes each coordinate to have zero mean and unit variance:

5.0 Content Block

Essentially, it is LayerNorm where we enforce $\mu, \epsilon = 0$. It is also called L2 normalization. It is a special case of L_p normalization, or power normalization: $\hat{x}_i = \frac{x_i}{\left(\frac{1}{D} \sum_{j=1}^D |x_j|^p\right)^{1/p}}, \quad y_i = \gamma \hat{x}_i + \beta$ where $p > 0$ is a constant.

6.0 Content Block

where $a_{x,y,i}$ is the activation of the neuron at location (x, y) and channel i . I.e., each pixel in a channel is suppressed by the activations of the same pixel in its adjacent channels.

7.0 Content Block

$$\mu_i(l) = \frac{1}{B} \sum_{b=1}^B x_{(b),i}(l), \quad \sigma_i(l)^2 = \frac{1}{B} \sum_{b=1}^B (x_{(b),i}(l) - \mu_i(l))^2$$

8.0 Content Block

$$\mu = \alpha E[x] + (1 - \alpha) \mu_{\text{train}}, \quad \sigma^2 = (\alpha E[x^2] + (1 - \alpha) \mu_{\text{train}}^2) - \mu^2$$

9.0 Content Block

$$x_{h,w,c}^{(l)} = \sum_{h',w',c'} K_{h'-h,w'-w,c,c'}^{(l)} x_{h',w',c'}^{(l-1)} + b_{c'}^{(l)}$$

10.0 Content Block

$$\begin{aligned} \hat{x}_{(b),h,w,c}^{(l)} &= x_{(b),h,w,c}^{(l)} - \mu_{c}^{(l)} (\sigma_{c}^{(l)})^2 + \gamma y_{(b),h,w,c}^{(l)} \\ \hat{x}_{(b),h,w,c}^{(l)} &+ \beta_{c} \sqrt{\frac{x_{(b),h,w,c}^{(l)} - \mu_{c}^{(l)}}{\sigma_{c}^{(l)} + \epsilon}} y_{(b),h,w,c}^{(l)} + \gamma_{c} \hat{x}_{(b),h,w,c}^{(l)} + \beta_{c} \end{aligned}$$

11.0 Content Block

ghost batching: randomly partition a batch into sub-batches and perform BatchNorm separately on each; weight decay on γ and β ; and combining BatchNorm with GroupNorm. A particular problem with BatchNorm is that during training, the mean and variance are calculated on the fly for each batch (usually as an exponential moving average), but during inference, the mean and variance were frozen from those calculated during training. This train-test disparity degrades performance. The disparity can be decreased by simulating the moving average during inference:

12.0 Content Block

where $\bar{a}_{x,y}^i$ is the average activation in a small window centered on location (x,y) and channel i . The hyperparameters k, n, α, β , and the size of the small window, are picked by using a validation set. Similar methods were called divisive normalization, as they divide activations by a number depending on the activations. They were originally inspired by biology, where it was used to explain nonlinear responses of cortical neurons and nonlinear masking in visual perception. Both kinds of local normalization were obviated by batch normalization, which is a more global form of normalization. Response normalization reappeared in ConvNeXT-2 as global response normalization.

13.0 Content Block

$b_{c'}^{(l)}$ is the bias term for the c' -th channel of the l -th layer. In order to preserve the translational invariance, BatchNorm treats all outputs from the same kernel in the same batch as more data in a batch. That is, it is applied once per kernel c (equivalently, once per channel c), not per activation $x_{h,w,c}^{(l+1)}$:

14.0 Content Block

$K_{\Delta h, \Delta w, c, c'}^{(l)}$ is a kernel tensor. Each channel c corresponds to a kernel $K_{h'-h, w'-w, c, c'}^{(l)}$, with indices $\Delta h, \Delta w, c'$.